

Complete SARS-CoV-2 Mutation Analysis Pipeline

Get sequence from :

https://www.ncbi.nlm.nih.gov/labs/virus/vssi/#/virus?SeqType_s=Nucleotide

Section 1: File Upload & Dependency Installation

Uploads FASTA files in Google Colab and installs Biopython for sequence parsing.

Code:

```
from google.colab import files
uploaded = files.upload()

!pip install biopython

from google.colab import files
uploaded = files.upload()

from Bio import SeqIO

records = list(SeqIO.parse("sequences (2).fasta", "fasta"))
print("Total sequences:", len(records))
print("Example length:", len(records[0].seq))
```

Section 2: Multiple Sequence Alignment (MAFFT)

Installs MAFFT and performs multiple sequence alignment for genomic comparison.

Code:

```
!apt-get -qq install mafft
!mafft --auto "sequences (2).fasta" > aligned.fasta

from Bio import SeqIO
aligned = list(SeqIO.parse("aligned.fasta", "fasta"))

print("Aligned sequences:", len(aligned))
print("Aligned length:", len(aligned[0].seq))
```

Section 3: Reference Genome Integration

Uploads reference genome (NC_045512), combines with samples, and realigns all sequences together.

Code:

```

from google.colab import files
uploaded = files.upload()

ref = SeqIO.read("reference_genome.fasta", "fasta").seq
print("Reference length:", len(ref))

with open("combined.fasta", "w") as outfile:
    outfile.write(open("reference_genome.fasta").read())
    outfile.write(open("sequences (2).fasta").read())

!mafft --auto combined.fasta > combined_aligned.fasta

aligned = list(SeqIO.parse("combined_aligned.fasta", "fasta"))
print("Total aligned sequences:", len(aligned))

```

Section 4: Reference & Sample Separation

Separates aligned reference genome from sample genomes for mutation comparison.

Code:

```

for i, record in enumerate(aligned[:5]):
    print(i, record.id)

ref_aligned = None
samples = []

for record in aligned:
    if "NC_045512" in record.id and ref_aligned is None:
        ref_aligned = record.seq
    elif "NC_045512" not in record.id:
        samples.append(record)

print("Reference set:", ref_aligned is not None)
print("Number of samples:", len(samples))

```

Section 5: Mutation Detection

Compares each sample to the reference and records mutations in format RefBase + Position + AltBase (e.g., A23403G).

Code:

```

mutation_data = []

for record in samples:
    seq = record.seq
    mutations = []

```

```

    for i in range(len(ref_aligned)):
        if ref_aligned[i] != seq[i] and seq[i] not in ["-", "N"]:
            mutations.append(f"{ref_aligned[i]}{i+1}{seq[i]}")

    mutation_data.append({
        "id": record.id,
        "mutations": mutations
    })

print("Number of mutations in first real sample:",
      len(mutation_data[0]["mutations"]))
print("First 10 mutations:", mutation_data[0]["mutations"][:10])

```

Section 6: Mutation Frequency Analysis

Counts frequency of each mutation across all samples.

Code:

```

from collections import Counter

all_mutations = []

for sample in mutation_data:
    all_mutations.extend(sample["mutations"])

mutation_counts = Counter(all_mutations)

print("Top 10 most common mutations:")
for mut, count in mutation_counts.most_common(10):
    print(mut, ":", count)

```

Section 7: Mutation Matrix Construction

Creates a binary matrix (samples × mutations) for machine learning.

Code:

```

import pandas as pd

unique_mutations = sorted(set(all_mutations))

df = pd.DataFrame(0,
                  index=[sample["id"] for sample in mutation_data],
                  columns=unique_mutations)

for sample in mutation_data:
    for mut in sample["mutations"]:
        df.loc[sample["id"], mut] = 1

```

```
print("Shape of mutation matrix:", df.shape)
df.head()
```

Section 8: PCA Dimensionality Reduction

Reduces mutation matrix to 2 principal components for visualization.

Code:

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2)
X_pca = pca.fit_transform(df)

plt.figure(figsize=(6,5))
plt.scatter(X_pca[:,0], X_pca[:,1])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA of SARS-CoV-2 Mutation Profiles")
plt.show()
```

Section 9: K-Means Clustering

Clusters genomes into 3 mutation-based groups.

Code:

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(df)

plt.figure(figsize=(6,5))
plt.scatter(X_pca[:,0], X_pca[:,1], c=labels)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Clustered SARS-CoV-2 Genomes")
plt.show()

import numpy as np
print(np.bincount(labels))
```

Section 10: Signature Mutation Detection

Identifies mutations present in >80% of genomes within each cluster.

Code:

```
df_clustered = df.copy()
df_clustered["cluster"] = labels

signature_mutations = {}

for cluster in np.unique(labels):
    cluster_samples = df_clustered[df_clustered["cluster"] == cluster]
    freq = cluster_samples.drop("cluster", axis=1).mean()
    sig = freq[freq > 0.8].index.tolist()
    signature_mutations[cluster] = sig[:10]

for cluster, muts in signature_mutations.items():
    print(f"Cluster {cluster} signature mutations:")
    print(muts)
```

Section 11: Gene Mapping

Maps mutation genomic positions to SARS-CoV-2 genes.

Code:

```
def map_gene(position):
    if 266 <= position <= 21555:
        return "ORF1ab"
    elif 21563 <= position <= 25384:
        return "Spike"
    elif 25393 <= position <= 26220:
        return "ORF3a"
    elif 26245 <= position <= 26472:
        return "Envelope"
    elif 26523 <= position <= 27191:
        return "Membrane"
    elif 28274 <= position <= 29533:
        return "Nucleocapsid"
    else:
        return "Other"

for cluster, muts in signature_mutations.items():
    for mut in muts[:5]:
        pos = int(mut[1:-1])
        print(mut, "→", map_gene(pos))
```

Section 12: Metadata Extraction (Year)

Extracts year from FASTA headers using regex.

Code:

```
import re

metadata = []

for record in samples:
    desc = record.description
    match = re.search(r"(20\d{2})", desc)
    year = int(match.group(1)) if match else None

    metadata.append({
        "id": record.id,
        "year": year
    })

metadata_df = pd.DataFrame(metadata)
print(metadata_df["year"].value_counts())
```

Section 13: Emerging Mutation Analysis (2023 vs 2026)

Detects mutations increasing between 2023 and 2026.

Code:

```
df_time = df.copy()
df_time["year"] = metadata_df.set_index("id").loc[df.index]["year"]

df_filtered = df_time[df_time["year"].isin([2023, 2026])]

mutation_shift = {}

for mut in unique_mutations:
    freq_2023 = df_filtered[df_filtered["year"] == 2023][mut].mean()
    freq_2026 = df_filtered[df_filtered["year"] == 2026][mut].mean()
    mutation_shift[mut] = freq_2026 - freq_2023

emerging = sorted(mutation_shift.items(), key=lambda x: -x[1])
print(emerging[:10])
```

Section 14: Random Forest Classification

Builds ML model to classify genomes by year.

Code:

```

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

X = df_filtered.drop("year", axis=1)
y = df_filtered["year"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))

```

Section 15: Mutation Burden Over Time

Calculates average mutation count per genome per year.

Code:

```

df_time = df.copy()
df_time["year"] = metadata_df.set_index("id").loc[df.index]["year"]

df_time["mutation_count"] = df_time.drop("year", axis=1).sum(axis=1)

df_time = df_time[df_time["year"] >= 2019]
df_time = df_time[df_time["mutation_count"] < 300]

mutation_per_year = df_time.groupby("year")["mutation_count"].mean()

import matplotlib.pyplot as plt
mutation_per_year.plot(kind="bar")
plt.xlabel("Year")
plt.ylabel("Average Mutation Count")
plt.title("Average Mutation Burden per Year")
plt.show()

```